

FeedbackHub SRS

FeedbackHub — Software Requirements Specification

What this file is. The full study of the job: every actor, every rule, every screen, the stack, the architecture, the data, and the tests.

Where it comes from. [references/FeedbackHub-Assignment.pdf](#) . If this file and the brief disagree, the brief wins.

State. Nothing is built yet. This says what must be true, not what is done. [README.md](#) is the file that tells the truth about what works.

Words used. **MUST** — the brief asks for it. **GOOD** — makes it better, can be cut. Every rule has a number (R-1, R-2, ...) so we can point at it later.

Size we plan for. One company, one board. A few hundred people, a few thousand requests, a few tens of thousands of votes. Every speed number in part 16 is for this size.

Contents

1 What the app is	10 Pictures
2 The hard parts	11 Settings and where they live
3 Who uses it	12 The data
4 Who can do what	13 The ERD
5 Use case picture	14 Rules that are always true
6 The journeys from the brief	15 Every case on every screen
7 The screens	16 Things that are not features
8 The stack and the architecture	17 How we know a feature is done
9 Rules, feature by feature	

1. What the app is

FeedbackHub is a feedback board for the people inside one company.

- People write feature requests and feedback.
- Everyone can read them, search them, and vote for the ones they like.
- People talk about a request in comments.
- An admin sets the status of a request, keeps the category list tidy, removes bad comments, and changes the settings of the app.

Why it exists, in the words of the brief: stop the same idea coming five times by email, and show what is really being worked on.

Two things follow from that one sentence, and we treat them as real work, not extras:

1. Finding an old request must be easy — so, search and filters.
2. The status must be easy to see everywhere a request is shown.

2. The hard parts (read this first)

Some parts of this job are easy to get wrong, and the brief says they carry the most marks.

#	The hard part	Why it is easy to get wrong
H-1	The AI write-up	The brief says it carries the most weight. It cannot be written at the end from memory. The prompts and the mistakes must be saved while working.
H-2	Checks on the server, not in the screen	Hiding a button is not a check. If someone calls the server directly, the server must still say no.
H-3	Loading, empty and error states	The brief says these are part of the work, not polish for later. They are the first thing dropped when time is short.
H-4	One call at start, not a chain	The brief asks for this by name. Four or five calls waiting on each other is the wrong answer.
H-5	A feature switch that really changes the app	Hiding a button is not a feature switch. The server must refuse the action too.
H-6	Categories and statuses are data	If they are fixed in the code, the admin cannot change them, and the brief says the admin must.
H-7	One vote per person	Easy to say, easy to break with a double click. The database must stop it, not the code.
H-8	One command starts everything, with data	"It works on my computer" fails on the reviewer's. It must be tested from a fresh clone.
H-9	The same style everywhere	When code is written in bursts, the same job ends up done three different ways.
H-10	Saying what does not work	Cheap to do, and the brief gives marks for it.

What this means for the order of work. H-2, H-3 and H-9 all say the same thing: build the shared parts first — the permission check, the error shape, the loading and empty and error boxes — then build features on top of them.

3. Who uses it

Who	Type	What they are
Visitor	Person	Not signed in. Can only see the sign-in page.
User	Person	A signed-in person from the company. Writes, reads, votes, comments. Owns their own content and their own settings.
Admin	Person	Someone from the product team. Everything a User can do, plus set statuses, tidy the categories, remove bad comments, and change app settings.
Identity provider	System	An outside open-source system. It owns the passwords, the sign-in page, the sign-up page, and social sign-in. We never write these ourselves.
Mail sender	System	Sends the emails people asked for, over SMTP, in a background job, after the action is saved.

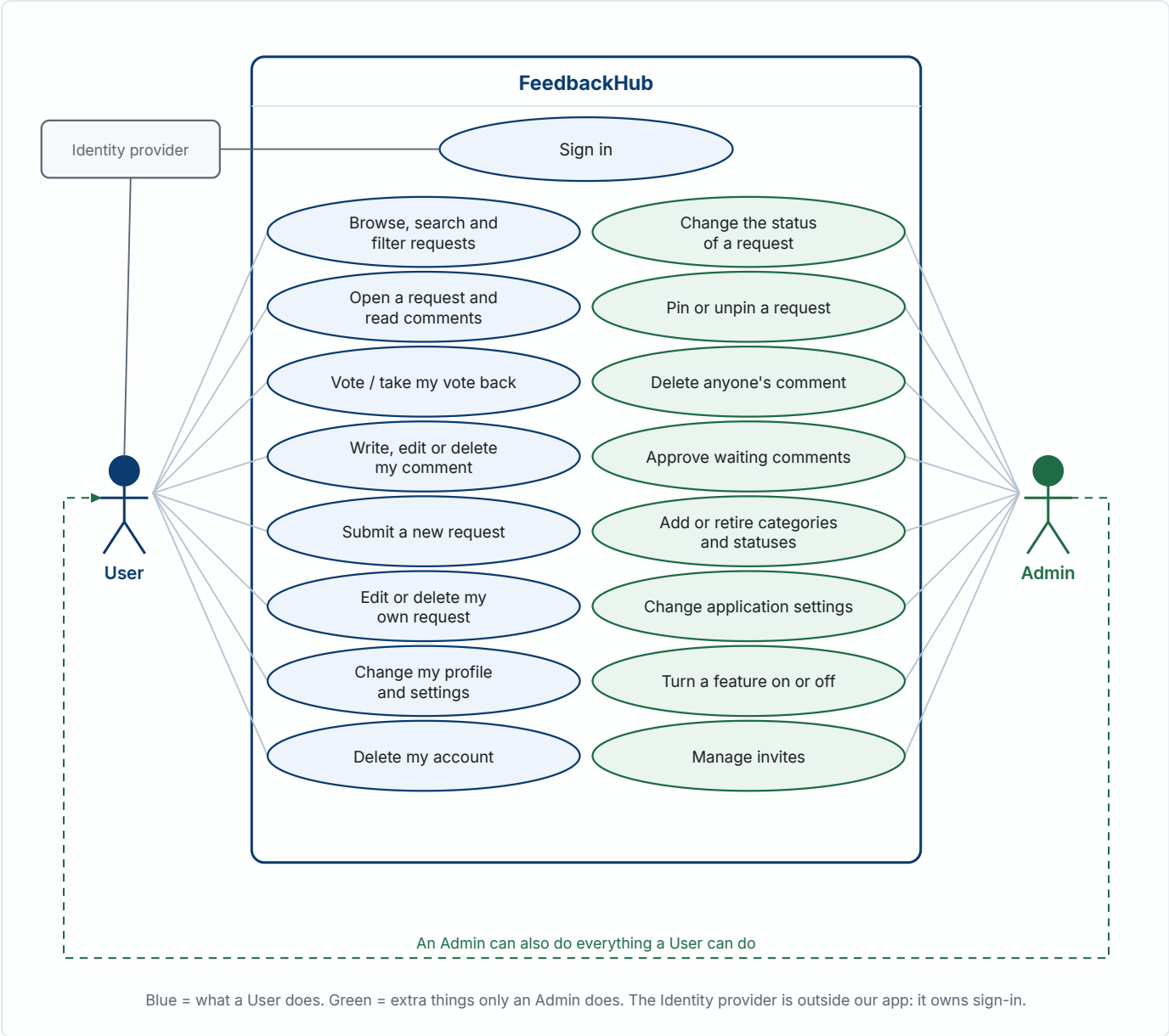
4. Who can do what

This table is a rule, not a description. Every line is checked on the server. Every **no** gets a test that proves the server says no.

Action	Visitor	User	Admin
See the board and open a request	no	yes	yes
Write a new request	no	yes, up to the limit	yes, up to the limit
Edit a request (title, text, category)	no	only mine	any
Delete a request	no	only mine	any
Change the status of a request	no	no	yes
Pin or unpin a request	no	no	yes
Vote, or take my vote back	no	yes, even on my own request, up to the limit	yes, up to the limit
Write a comment	no	yes, if comments are switched on	same
Edit a comment	no	only mine	only mine. An admin removes a bad comment; an admin never rewrites what someone said.
Delete a comment	no	only mine	any
Approve a waiting comment	no	no	yes

Action	Visitor	User	Admin
Add or retire a category or status	no	no	yes
Change app settings and feature switches	no	no	yes
Manage invites	no	no	yes
Change my own profile and settings	no	yes	yes
Delete my own account	no	yes	yes, unless last admin (R-62)
See another person's email or settings	no	no	no — nothing in the brief needs it

5. Use case picture



6. The journeys from the brief

The brief lists eleven journeys. Every one is here, cut into steps, with the rules that make each step work.

6.1 User journeys

#	From the brief	What must happen	Rules
U-1	Registers or signs in through the identity provider.	The person is sent to the identity provider. They sign in with email and password, or with a social account. They come back signed in. The first time only, we make a local record — but only if the sign-up rule and the sign-up limit allow it.	R-1...R-9

#	From the brief	What must happen	Rules
U-2	Lands on the request list: sorted, filterable by status and category, searchable by text, paginated.	The board opens. The person can sort, filter by status, filter by category, search words, and move page by page. Their own saved sort and filters are used the first time.	R-16... R-25
U-3	Opens a request, reads the discussion, upvotes it, adds a comment.	The request page shows the text, who wrote it, the status, the votes, and the comments. The vote button works with one click and with the keyboard. The comment box adds a comment without reloading.	R-13, R-26...R-42
U-4	Submits a new request; sees it appear in the list.	A form with title, text and category. The server sets the status. After saving, the person goes to their new request and it is on the board.	R-10... R-14
U-5	Finds their own earlier request, edits its description, and later deletes it.	The person can find their own requests, change the text, and later delete it after a clear question that says what will be lost.	R-13, R-14
U-6	Updates their profile and application preferences.	A settings screen: name, picture or initials, theme, language, default sort, default filters, email choices, and delete my account.	R-54... R-62

6.2 Admin journeys

#	From the brief	What must happen	Rules
A-1	Signs in and reviews recently submitted requests.	The board, newest first, filtered to the first status. One saved link, not a new screen.	R-63
A-2	Changes a request's status and pins an important one to the top.	Status can be changed from the board and from the request page. A pinned request goes to the top.	R-64, R-65, R-23
A-3	Deletes an inappropriate comment.	The comment is replaced by a small grey line saying it was deleted. The thread does not break.	R-38, R-41
A-4	Adds a new category and retires an unused one.	Add, rename, recolour. Retire means hide from the picker — old requests still show it correctly. Deleting one that is in use is refused.	R-43...R-49
A-5	Adjusts application-wide settings.	Sign-up rule, comment approval, the three rate limits, and the comments switch. It works right away, with no restart.	R-67...R-70, R-130...R-132

7. The screens

Screen	Who	What is on it
Sign in	Visitor	Owned by the identity provider. We only send the person there and take them back.
Board (home)	User, Admin	Search box, status filter, category filter, sort, the list, page buttons. Pinned requests at the top. An admin can also change the status from here.

Screen	Who	What is on it
Request page	User, Admin	Title, text, category, status, who wrote it, when, vote button, the comment list, the comment box. Edit and delete for the owner or an admin.
New request	User, Admin	Title, text, category.
Edit request	Owner, Admin	Same fields as new. Not the status.
My settings	User, Admin	Name, picture, theme, language, default sort, default filters, email choices, delete my account.
Admin: categories and statuses	Admin	Add, rename, colour, retire. Shows how many requests use each one.
Admin: app settings	Admin	Sign-up rule, allowed email domains, comment approval, the three rate limits, and the comments switch.
Admin: waiting comments	Admin	Only when comment approval is on. Approve or reject.
Admin: invites	Admin	Only when the sign-up rule is invite-only. Add an email, see if it was used, copy the sign-up link.
Not allowed / Not found	All	Clear message, and a way back.

8. The stack and the architecture

The brief asks for the choices **and** the reasoning. The reasoning in full lives in [DECISIONS.md](#) ; this part states what we build and the rules that come with it.

8.1 The choices at a glance

Layer	Choice	The one-line reason
Front end	Angular 20, standalone components, signals	The brief names Angular. 20 is the current stable line: standalone components with no NgModules, signals for state, and the new control flow.
Front-end state	Signals + a small store service per feature	The state here is small. A signal store is a few lines and no extra library; NgRx would be more machinery than this app has state.
Styling	Tailwind CSS + a small set of our own components	Tokens for colour, spacing and dark mode in one place, so every screen looks the same (H-9). No heavy component kit to fight for keyboard and contrast.

Layer	Choice	The one-line reason
Back end	Node.js 22 with NestJS 11, TypeScript strict	The brief prefers Node. Nest gives one agreed shape for modules, dependency injection and pipes, which is exactly what H-9 asks for.
Architecture	Modular monolith: one API deployable, hard module seams, Clean Architecture inside each module. One extra worker process for email.	See 8.4.
Database	PostgreSQL 16	Relational data with real invariants — one vote per person, one first status, links that must not break. Postgres enforces all three (R-115).
ORM	Prisma 6, with raw SQL where it earns it	Typed schema, generated types, a real migration tool for R-82. Raw SQL for the one query where the ORM would be worse: the board.
Shared state	Redis 7	The rate-limit counters, idempotency keys and short-lived caches. Nothing is kept inside the API process (R-118, R-141).
Sign-in	Keycloak (open source, self-hosted in this repository)	Email + password and social sign-in out of the box, standard OIDC, runs from a container with our seed realm. We write no auth primitives.
Mail	SMTP, with Mailpit locally	Real sending, no queue table (R-73, R-129).
Packaging	Docker, Docker Compose for one-command start, Kubernetes manifests for a local cluster (kind)	The brief asks for containerized, environment-driven, orchestrator-ready.
Tests	Jest, Testcontainers, Supertest, Angular Testing Library, Playwright	See 8.9.
API document	OpenAPI, generated from the Nest decorators	The brief calls it optional; it is nearly free here.

8.2 The front end

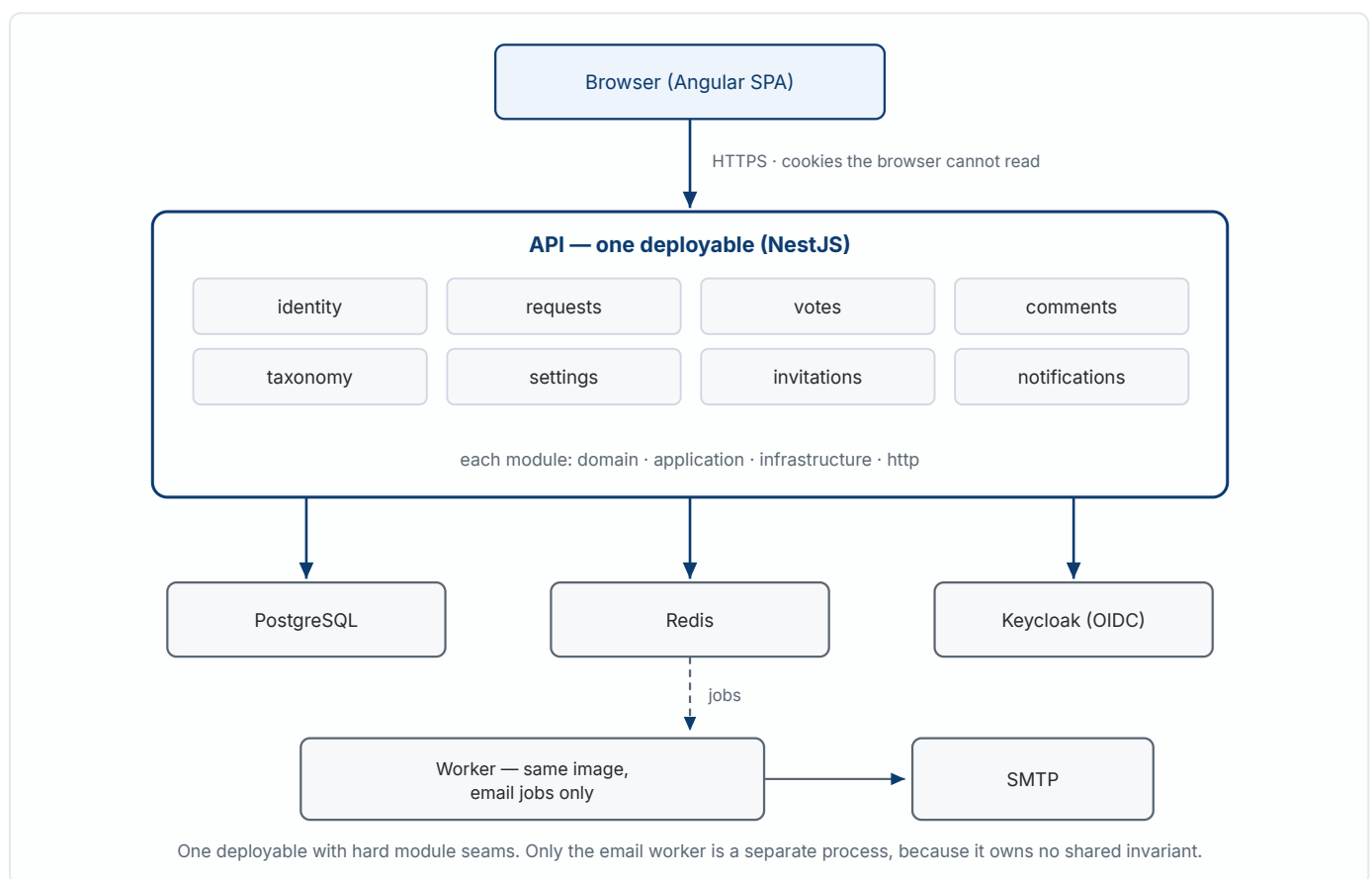
#		Rule
R-133	MUST	Angular 20 with standalone components. No NgModules. Routing is lazy per feature, so the board does not carry the admin screens.
R-134	MUST	State lives in signals. Server data is fetched through one typed API client per feature and held in a small store service. No component talks to <code>HttpClient</code> directly.
R-135	MUST	One design token file (colour, spacing, radius, font) drives Tailwind. Dark mode is a token swap, not a second stylesheet.
R-136	MUST	Loading, empty, error and not-allowed are four shared components, written once and used on every screen (see R-85).

8.3 The back end

#		Rule
R-137	MUST	Node.js 22 and NestJS 11, TypeScript with <code>strict</code> on. No <code>any</code> at any edge of the app (R-122).
R-138	MUST	Every request that changes data passes one guard chain in the same order: signed in → Origin check → schema check → permission check on the saved row → rate limit → the use case. The order is written once and never rebuilt per controller.
R-139	MUST	The HTTP layer is thin. A controller reads the request, calls one use case, and maps the result to a response DTO. No business rule lives in a controller.

8.4 The architecture

What we build: a modular monolith with hard seams, plus one worker.



Why one deployable and not several services. The brief says a well-reasoned case for simplicity beats an unjustified split, so here is the case:

- The rules that matter are transactional and cross two tables at once — count and write inside one database step (R-132), one vote per person (R-26), one first status (R-47). Split those across services and we would have to rebuild in application code what one Postgres transaction already gives us for free.
- One team, one release, a few thousand rows. A network hop between modules would buy latency and failure modes, and pay back nothing.
- The seams are what actually matter, and we can have them without separate processes. See below.

What we split off, and why. Sending email is the one job that must never slow down or break what the person just asked for (R-72, R-116). It runs in a separate worker process from the same image, reading jobs from Redis. It is also the first module we would extract into its own service if the app grew — it owns no shared invariant.

#		Rule
R-140	MUST	The API is one deployable, cut into modules: <code>identity</code> , <code>requests</code> , <code>votes</code> , <code>comments</code> , <code>taxonomy</code> , <code>settings</code> , <code>invitations</code> , <code>notifications</code> . Every module has its own folder, its own tables, and one published application service.
R-141	MUST	A module never reads or writes another module's tables and never imports another module's internals. It calls the other module's published service. This is the seam that makes extraction possible later.
R-142	MUST	Inside a module the layers are <code>domain</code> , <code>application</code> , <code>infrastructure</code> , <code>http</code> , and dependencies only ever point inward. <code>domain</code> imports no framework and no ORM. <code>application</code> defines the ports (repository interfaces); <code>infrastructure</code> implements them. A Prisma type never appears in <code>domain</code> or in an HTTP response.
R-143	MUST	The dependency rule and the module seams are checked by a tool in CI (ESLint import boundaries and dependency-cruiser), not by good manners. A pull request that crosses a seam fails.
R-144	MUST	The worker runs the same container image as the API, with a different command. It shares the code and the database, and it serves no HTTP.

8.5 The database and the ORM

#		Rule
R-145	MUST	PostgreSQL 16. Every promise in part 14 marked "Database" is a real constraint — a unique index, a foreign key, a partial unique index, a check — not code that hopes to run first.
R-146	MUST	Prisma owns the schema and the migrations. The schema file is the one source of truth for the tables in part 12, and every change ships as a migration (R-82).
R-147	MUST	Repository interfaces live in <code>application</code> ; the Prisma implementations live in <code>infrastructure</code> . Use cases and the domain never see a Prisma client, so the ORM can be swapped without touching a rule.
R-148	MUST	The board query is written as one SQL statement through Prisma's typed raw query: search, filters, sort, pinned-first, page and the two derived counts. Doing it in the ORM's object API would mean several round trips or a query we cannot read. Every value in it is a bound parameter (R-97).
R-149	MUST	Redis holds only shared state that must survive nothing: rate-limit counters (R-130), idempotency keys for double submits (R-117), and the bootstrap cache (R-52) with a short life. Losing Redis must degrade the app, never corrupt it — a lost counter means one person gets one extra try, and that is acceptable.

8.6 Clean code, at senior level

The brief says consistency is assessed. These are the rules that make it real.

#		Rule
R-150	MUST	One way to do each thing: one error shape, one validation library, one permission check, one pagination shape, one loading component. Decided once, then reused (H-9).
R-151	MUST	One use case per file, one public method, named after what the person does — <code>SubmitRequest</code> , <code>WithdrawVote</code> , <code>ApproveComment</code> . A use case reads as the story of the rule it implements.
R-152	MUST	Business rules live in the domain and are testable with no database, no HTTP and no clock. Time and randomness come in as ports.
R-153	MUST	Names say what a thing is, not what it is made of. No <code>utils</code> , no <code>helpers</code> , no <code>manager</code> , no <code>data</code> in a name.
R-154	MUST	Every function has a return type. <code>strict</code> is on, and <code>any</code> is banned by lint at every edge of the app (R-122).
R-155	MUST	ESLint and Prettier run in CI and fail the build. Commit messages follow Conventional Commits, and the history is real — the brief asks for it, so no squashing the whole job into one commit.

8.7 How we test: TDD

TDD is the working method, not a stage at the end. Every numbered rule in this file is turned into a failing test first, then the code that makes it pass, then the clean-up. Part 17 is the list of those tests.

#		Rule
R-156	MUST	Red, green, refactor. No production code is written without a test that fails first for the rule it serves.
R-157	MUST	Every no in part 4 has a test that calls the API directly, with no screen involved, and proves the server answers 401 or 403 and that nothing changed.
R-158	MUST	Four layers of tests, and each layer tests what only it can: Unit (Jest) — the domain and the use cases, with fake ports, no database. Fast, and the biggest group. Integration (Jest + Testcontainers) — the repositories and the database constraints against a real Postgres and a real Redis. This is where "the database stops it" is proved. API (Supertest against the running Nest app with a real database) — the guard chain, the error shape, the status codes. Front end (Jest + Angular Testing Library) — components through what a person sees and does, never through internals.
R-159	MUST	End-to-end tests with Playwright, against the whole stack from <code>docker compose</code> : real Angular build, real API, real Postgres, real Keycloak, Mailpit for mail. They cover the eleven journeys of part 6, and the two that prove the hard parts — one call at start-up (H-4) and the comments switch blocking the server (H-5).
R-160	MUST	End-to-end tests sign in for real through Keycloak with a seeded account. Mocking sign-in in an end-to-end test would remove the one thing it is there to prove.
R-161	MUST	Every fixed bug gets a test that fails without the fix.
R-162	MUST	CI runs lint, the dependency-rule check, all four test layers, the end-to-end suite, and a build of both images, on every push. A red pipeline is never merged.

#		Rule
R-163	GOOD	Accessibility is checked in the end-to-end run with an automated axe pass on the board, the request page and the settings screen. Automated checks are a floor, not a pass mark (R-108).

9. Rules, feature by feature

"The server" means the back end. When a rule says the server, the check happens there, and hiding it in the screen is not enough.

9.1 Sign in and accounts

#		Rule
R-1	MUST	All sign-in is done by the outside identity provider. We never store passwords, never hash them, never write a password-reset flow, never write the social sign-in dance.
R-2	MUST	A person can sign in with email and password, or with at least one social account. Both screens belong to the identity provider.
R-3a	MUST	The sign-in handshake happens on our server, not in the browser. The server sends the person to the identity provider, takes the code back, and swaps it for the tokens. The browser only follows redirects.
R-3b	MUST	The swap uses code + PKCE. Our client at the identity provider is a confidential one. Its secret lives only on the server, in an environment variable — never in the front-end build, never in the repository.
R-3c	MUST	The browser never receives a token. Not in a page, not in a body, not in a header, not in a URL. The access token and the refresh token travel only as cookies the browser cannot read.
R-3d	MUST	Every auth cookie is HttpOnly and Secure. No <code>Domain</code> is set, so no other subdomain can read it.
R-3e	MUST	The refresh cookie is scoped <code>Path=/auth/refresh</code> and <code>SameSite=Strict</code> . It is sent to that one endpoint and nowhere else. The access cookie is <code>Path=/</code> and <code>SameSite=Lax</code> .
R-3f	MUST	Cookie names and lifetimes are never written in the code. They come from config — the table below.
R-3g	MUST	Because cookies go with every request, the server stops cross-site requests two ways: <code>SameSite</code> on the cookie, and an Origin check on the server for every request that changes data. A request that changes data is never a GET.
R-3h	MUST	The front end and the API are served from the same site. If they are ever split across two sites, <code>SameSite</code> stops working and this rule fails.
R-4	MUST	The first time a person signs in, the server makes a local record — but only after checking the sign-up rule (R-67) and that the sign-up limit has room (R-130). If either says no, no record is made.
R-5	MUST	The server checks every token: the signature, who made it, who it is for, and that it is not old. It never trusts a token it did not check. The token is read from the cookie, never from a header.
R-6	MUST	Every part of the app needs a signed-in person, except the health checks and the sign-in page. Not signed in → 401. Signed in but not allowed → 403. These two are never mixed up.

#		Rule
R-7	MUST	The server decides "is this allowed" by reading the saved row from the database — never by trusting an id or a role sent in the request.
R-8	MUST	Being an admin is read from the checked token and compared with the local record before any admin action.
R-9	MUST	Sign out clears the app and ends the session at the identity provider too.
R-9a	MUST	We keep no session list on the server, so we cannot end one session from outside. Instead: the access token lives 5 minutes, the refresh token 30 minutes, and the identity provider rotates the refresh token on every use. Sign out also calls the identity provider, so its session ends. That is our only way to cut a session short.

The config for R-3f. Every one is an environment variable.

Variable	What it is	Default
<code>AUTH_COOKIE_ACCESS_NAME</code>	Name of the access token cookie	<code>at</code>
<code>AUTH_COOKIE_REFRESH_NAME</code>	Name of the refresh token cookie	<code>rt</code>
<code>AUTH_COOKIE_ACCESS_MAX_AGE</code>	How long the access cookie lives, in seconds	<code>300</code>
<code>AUTH_COOKIE_REFRESH_MAX_AGE</code>	How long the refresh cookie lives, in seconds	<code>1800</code>
<code>AUTH_COOKIE_REFRESH_PATH</code>	Path the refresh cookie is scoped to	<code>/auth/refresh</code>
<code>AUTH_COOKIE_SECURE</code>	Send the cookie only over HTTPS. False on localhost only.	<code>true</code>
<code>AUTH_ALLOWED_ORIGINS</code>	Origins the server accepts writes from (R-3g)	—

The two `MAX_AGE` values must match the token lifetimes set in the identity provider. If they drift, a cookie outlives its token, or dies before it. `AUTH_ALLOWED_ORIGINS` has no default: if it is empty the server refuses to start. An empty list must never quietly mean "allow everything".

9.2 Writing and changing a request

#		Rule
R-10	MUST	A request has a title, a description and a category. The person picks these three. The server sets the status, the author, the time, and the counts.
R-11	MUST	The server gives every new request the status marked as the first one. A person can never choose or send a status.
R-12	MUST	Checks, on the server and in the form: title 5 to 120 letters, description 10 to 5000 letters, the category must exist and be active. A bad value gets a message under the field that says how to fix it.
R-13	MUST	The person who wrote it can change the title, text and category. An admin can change any request.
R-14	MUST	The person who wrote it can delete it. An admin can delete any. Deleting removes its votes and its comments too. We ask first, and the question says what will be lost.

9.3 The board

#		Rule
R-16	MUST	The board shows: title, category, status, who wrote it, number of votes, number of comments and a pin mark.
R-17	MUST	Search by words, over the title and the description. Not over comments.
R-18	MUST	Filter by status and by category. More than one can be picked in each. Inside one filter it means "or"; between the two filters it means "and".
R-19	MUST	Sort by: newest, oldest, most votes, most comments.
R-20	MUST	The server only accepts sort names from a fixed list. Anything else is refused. A name from the user never goes into the database query as it is.
R-21	MUST	Pages. 20 per page by default, named <code>page</code> and <code>pageSize</code> . The answer says how many there are in total.
R-22	MUST	The search, filters, sort and page are in the web address, so a board view can be copied, shared and found again with the back button.
R-23	MUST	Pinned requests come first — but only inside the filter the person chose. Pinning must never show a request the filter says to hide, and never hide one the filter says to show.
R-24	MUST	On the first visit the board uses the person's saved sort and filters. If the web address says something else, the web address wins.
R-25	MUST	Four different states: loading, nothing here yet, nothing matches your filters (with a Clear button), and something went wrong (with a Retry button).

9.4 Votes

#		Rule
R-26	MUST	One person can vote for one request one time, and can take the vote back. The database stops a second vote, not the code.
R-27	MUST	Voting twice, or un-voting when there is no vote, is fine. It gives back the current state instead of an error. A double click can never make two votes.
R-28	MUST	The number of votes is counted by the server from the real votes. The browser can never send a number.
R-29	MUST	A person may vote for their own request. A new request starts at zero votes, not one.
R-30	GOOD	The number changes at once when clicked. If the server says no, it goes back and shows why.
R-31	MUST	The vote control is a real button: it works with the keyboard, its name says the count and whether you voted, and the change is read out by a screen reader.

9.5 Comments

#		Rule
R-32	MUST	Anyone signed in can comment on any request. 1 to 2000 letters.
R-33	MUST	Comments are a flat list, newest first. No replies inside replies.
R-33a	MUST	The order is <code>created_at</code> newest first, with <code>id</code> newest first as the tie-breaker. Two comments saved in the same moment must still come back in the same order every time, or a comment is shown twice or missed. The order is fixed in the server: there is no sort the person can pick.
R-33b	MUST	The list is read with a cursor, not with page numbers: <code>limit</code> , and <code>cursor</code> for the next block. The cursor is the <code>created_at</code> and <code>id</code> of the last row sent, and it means nothing to the browser. The answer gives back <code>nextCursor</code> (empty means the end) and the total. The board uses <code>page</code> and <code>pageSize</code> ; comments do not, on purpose — newest-first with page numbers shows the same comment twice whenever a new one arrives while a person is reading.
R-33c	MUST	The total is counted for the person asking. A waiting comment (R-40) is counted only for its writer and for admins, so two people can see two different totals on the same request. Both are right.
R-33d	MUST	A comment a person writes goes to the top of the list at once, with no reload and no second call.
R-34	MUST	Each comment shows who wrote it and when.
R-35	MUST	The writer can edit their own comment.
R-36	MUST	An admin cannot edit someone else's comment. Moderation means deleting, never changing words.
R-37	MUST	The writer can delete their own comment. An admin can delete any.
R-38	MUST	A deleted comment leaves a small grey line: "This comment was deleted." The row stays so the thread still makes sense. The text is gone for good.
R-39	MUST	Deleted comments are not counted and cannot be edited.
R-40	MUST	When the admin turns on <i>comments need approval</i> , a new comment waits. Only the writer (marked "waiting") and admins can see it. It is not counted until it is approved.
R-41	MUST	An admin can approve a waiting comment (it appears) or reject it (it becomes a deleted line).
R-42	MUST	When the comments feature switch is off: the comment box and the thread are gone, comment counts are gone from the board, and the server refuses to save a comment with a clear message. This is the switch that proves H-5.

9.6 Categories and statuses

#		Rule
R-43	MUST	Categories and statuses are rows in the database, not fixed words in the code. The admin changes them while the app runs.
R-44	MUST	The admin can add, rename and recolour them. Two cannot have the same name, even with different capital letters.
R-45	MUST	Retire means: gone from the picker, but still shown correctly on old requests that use it.
R-46	MUST	A category or status that is used by any request cannot be deleted. The database refuses it. Only retiring is possible.
R-47	MUST	Exactly one status is marked as the first one. Marking a new one un-marks the old one, in the same step.
R-48	MUST	The first status cannot be retired. The last active category cannot be retired. Otherwise nobody could write a request.
R-49	MUST	Both lists come with the one start-up call (R-52), in <code>created_at</code> order. They are not a separate call.

9.7 Where the settings live

The brief asks this by name: where configuration lives, how it is resolved between global defaults and user overrides, and how the front end obtains it without a chain of blocking requests on startup. This is hard part H-4.

#		Rule
R-51	MUST	There are two places, and one order: code default , then user setting . The last one filled in wins. For theme, default sort and default filters that last layer sits in the browser. Written one time, and tested.
R-52	MUST	The browser gets everything it needs to start — who I am, my language and email choices, the switches, the categories, the statuses — in one call, fetched on every load. A chain of calls is a bug.
R-53	MUST	Server addresses, database address and secrets come from the environment. They are never used to decide how the product behaves. Product settings never come from the environment.

9.8 My settings

#		Rule
R-54	MUST	A person can set their display name and a picture. With no picture we draw their initials. No file upload.
R-55	MUST	A person can set the theme: light, dark, or follow the computer.
R-56	MUST	The theme is kept in the browser and read before the first paint, so the next load is right with no white flash. "Follow the computer" changes live when the computer changes.
R-57	MUST	A person can set the language. It is kept on the server and copied into the browser. It changes at once, with no rebuild. A missing word shows in English, never as a code word.

#		Rule
R-58	MUST	A person can set their default sort and default filters. Both are kept in the browser.
R-59	MUST	A person can choose their email messages: when someone comments on my request, and when the status of my request changes.
R-60	MUST	A person can only change a fixed list of their own settings: language and email choices. Trying to change an admin setting, a theme or a sort through the API is refused with a message, not quietly ignored.
R-61	MUST	A person can delete their account. Their name, picture and email are wiped, their sign-in stops working, their votes go. Their requests and comments stay, shown as "Deleted user". The question we ask says this before they press the button.
R-62	MUST	The last admin cannot delete their own account. The app must never be left with nobody who can run it.

9.9 Admin jobs and the rate limits

#		Rule
R-63	MUST	An admin can see new requests: the board, newest first, filtered to the first status. One saved link is enough.
R-64	MUST	Only an admin changes a status. It can be done from the board and from the request page, and it shows at once.
R-65	MUST	Only an admin pins or unpins. More than one request can be pinned.
R-66	MUST	Only an admin can add, see or remove an invitation. The server reads the role from the saved row (R-7, R-8) before every one. A normal person who calls the invite endpoint by hand gets 403. Hiding the screen is not the check.
R-67	MUST	The admin sets the sign-up rule: open to everybody / invite only / only these email domains, with a list of domains. Only a checked email counts for the domain rule — an unchecked email is a claim, not a fact.
R-68	MUST	The admin can turn <i>comments need approval</i> on and off.
R-69	MUST	The admin sets all six rate-limit numbers while the app runs, with no restart. Admins are limited too: a limit is not a permission, and a stolen admin account should not be an open door.
R-70	MUST	Admin screens are hidden from normal people and refused by the server if someone types the address by hand. Every setting is checked when saved, and works with no restart. Deleting things always asks first and names the thing.
R-130	MUST	Three things are rate limited: making a new account, writing a new request, and voting. Each has its own count and its own window in <code>app_settings</code> . Requests and votes are counted for one person. New accounts are counted for the whole app, because at that moment there is no person yet.

#		Rule
R-131	MUST	Every window slides. It is not reset on the hour. The refusal says when the person may try again, and that time is one window after their oldest one, not one window from now. Nothing they typed is thrown away. A deleted request still counts while it is inside the window — otherwise write, delete, write again walks round the limit.
R-132	MUST	The count and the write happen inside one database step, so two calls in the same second cannot both slip through. This covers new requests, votes and new accounts.

The three limits.

Field	Default	What it means	Counted for
<code>signup_limit_count</code>	20	20 new accounts	the whole app
<code>signup_limit_minutes</code>	60	in 60 minutes	
<code>submission_limit_count</code>	10	10 new requests	one person
<code>submission_limit_minutes</code>	60	in 60 minutes	
<code>vote_limit_count</code>	100	100 votes or un-votes	one person
<code>vote_limit_minutes</code>	60	in 60 minutes	

Every field is needed and the smallest value is 1 — never empty, never 0, since 0 would mean nobody can write at all.

How the sliding window reads. Every time a person tries to write, the server asks one question: how many did this person already write in the last *minutes* minutes? If the answer is *count* or more, it says no.

Time	They try	The server counts the last 60 minutes	Result
13:00	request 1	0	saved
...	requests 2 to 10	under 10 each time	saved
13:40	request 11	10	refused
14:00	request 11 again	9 — the 13:00 one is too old now	saved

At 13:40 the message says "you can send another at 14:00". That is one hour after their oldest one, not one hour from now: they waited 20 minutes, not 60. Resetting on the hour would let someone send 10 at 13:59 and 10 more at 14:00, and we could not name a time to come back.

Why these three. New accounts, because R-4 makes a local record the first time someone signs in, and under the *open* sign-up rule nothing else stops a script filling the database with real accounts. Requests, because the brief asks for it by name. Votes, because a vote and an un-vote are both writes, and the one-vote rule (R-26) stops a second vote, not a hundred vote/un-vote pairs.

When the sign-up limit is full, the person has already signed in at the identity provider; we just do not make their record. The message must say to try later, not that they are not allowed — because they are. They were unlucky with the timing.

9.10 Emails

#		Rule
R-71	MUST	When something happens, we look at what the person chose, and send the email in the background. Nobody is told about their own action.
R-72	MUST	Writing that message can never break or slow down the thing the person just did. A failed email must never undo a comment.
R-73	MUST	The email is really sent, over SMTP. Three events only: a comment on my request, a status change on my request, and an invitation.
R-74	MUST	The email is sent after the action is saved, in a background job, never inside the request.
R-125	MUST	When comments need approval, the email is sent when the comment becomes visible, not when it is written. A comment the admin rejects is never emailed. When approval is off, the comment is visible at once and so is the email.
R-126	MUST	An invitation email needs no preference check. The invited person has no account and no settings yet.
R-127	MUST	A failed email is written to the log and dropped. There is no retry and no record of it. The README says this plainly.
R-128	MUST	Mail settings come from the environment: host, port, user, password, from address, timeout, and an on/off switch. Nothing is written into the code.
R-129	MUST	The one command in R-80 also starts a local mail catcher, so the whole system runs with no real mail server and the email can be read in a browser.

The mail settings (R-128).

Name	What it is
SMTP_HOST	The mail server.
SMTP_PORT	Normally 587.
SMTP_USER	The account name.
SMTP_PASSWORD	A secret. Never in the repository.
SMTP_FROM	The address the mail comes from.
SMTP_TIMEOUT	Seconds. Must be set — the library default is minutes, and that hangs the job.
MAIL_ENABLED	Off in development: the message goes to the log instead of the wire.

The worst case is a lost invitation: the person cannot sign up and nobody is told. To soften it, the admin screen shows the sign-up link next to each invitation, so it can be sent by hand.

9.11 The API, and running it

#		Rule
R-75	MUST	One web API with a version in the address. Normal status codes.

#		Rule
R-76	MUST	One shape for every error: a code the app turns into words in the person's language, an English message for the logs, the field names when a form is wrong, and an id we can search.
R-77	MUST	The API sends only the fields it means to send. We never send a database row straight out, or a new column leaks the day it is added.
R-78	GOOD	An API document generated from the code.
R-79	MUST	Every part runs in a container: small, built in steps, running as a normal user, not as root.
R-80	MUST	One written command starts everything — front end, API, worker, database, Redis, identity provider, mail catcher — with example data, from a fresh copy of the repository.
R-81	MUST	Files for a small local Kubernetes: how to run each part, how to reach it, settings and secrets kept apart, health checks, and limits.
R-82	MUST	Database changes run as their own step that finishes before the app starts. Never while the app is starting, because two copies would fight.
R-83	MUST	Two health checks: "is it alive" and "is it ready". Ready means the database, Redis and the identity provider can be reached.
R-84	MUST	The same container image runs everywhere. Only the environment values change.

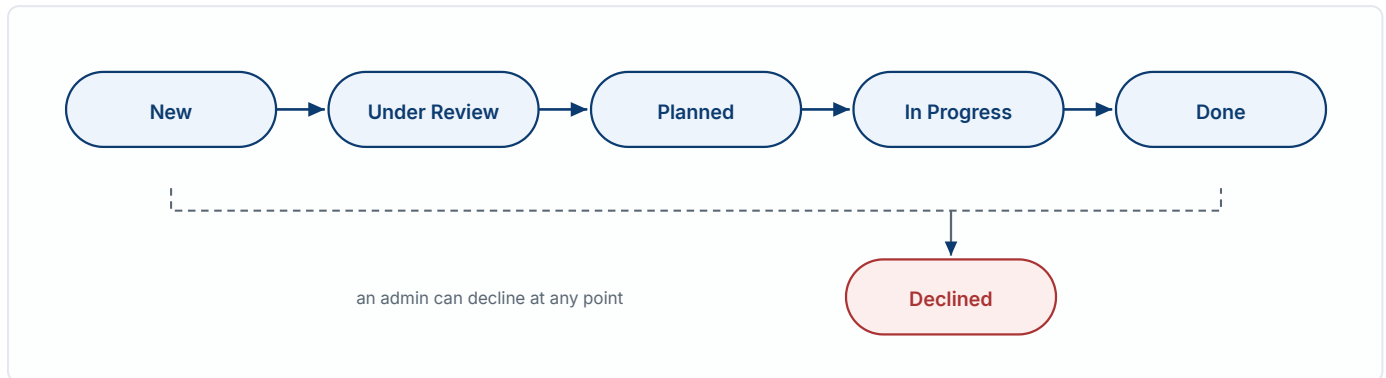
9.12 How it must feel to use

The brief says user experience is part of the work, not polish for later. So these are rules with tests, not wishes.

#		Rule
R-85	MUST	Every screen that loads data has: loading, empty, error, and where it makes sense not allowed. They are built one time and used everywhere (R-136).
R-86	MUST	While loading, the page keeps its shape — grey blocks the same size as the real thing. Nothing jumps when the data comes.
R-87	MUST	An error says what failed, whether trying again helps, and gives the button to try again. Never a raw error code or a stack trace.
R-88	MUST	A form message says how to fix it ("Title must be at least 5 letters"), sits next to the field, and shows when leaving the field or on save — not on every key press.
R-89	MUST	Everything works with the keyboard alone, in a sensible order, and you can always see where you are. A pop-up keeps the keyboard inside it and gives it back when it closes.
R-90	MUST	The layout works from a 360 pixel phone screen up. The page never scrolls sideways.
R-91	MUST	Anything that deletes asks first, names the thing, and says what will be lost. Nothing that cannot be undone happens on one click.
R-92	GOOD	Things that finish in the background — vote saved, comment added, settings saved, an error — are read out for screen readers.

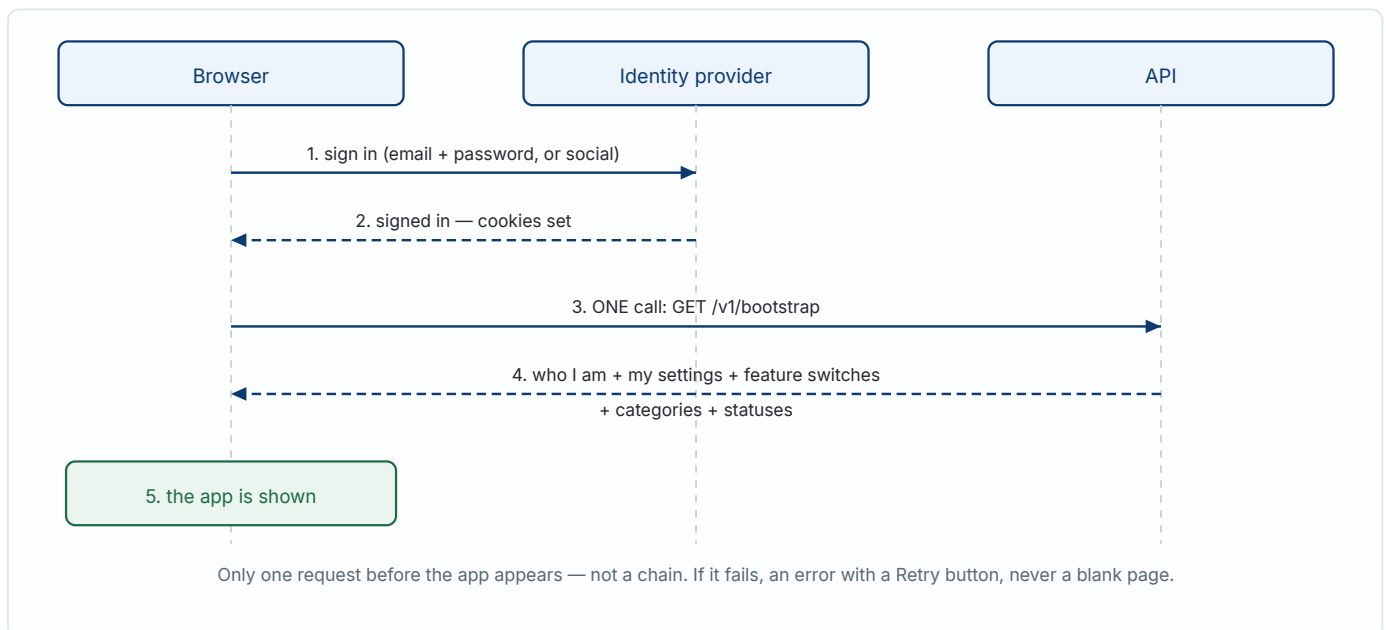
10. Pictures

10.1 How a request moves



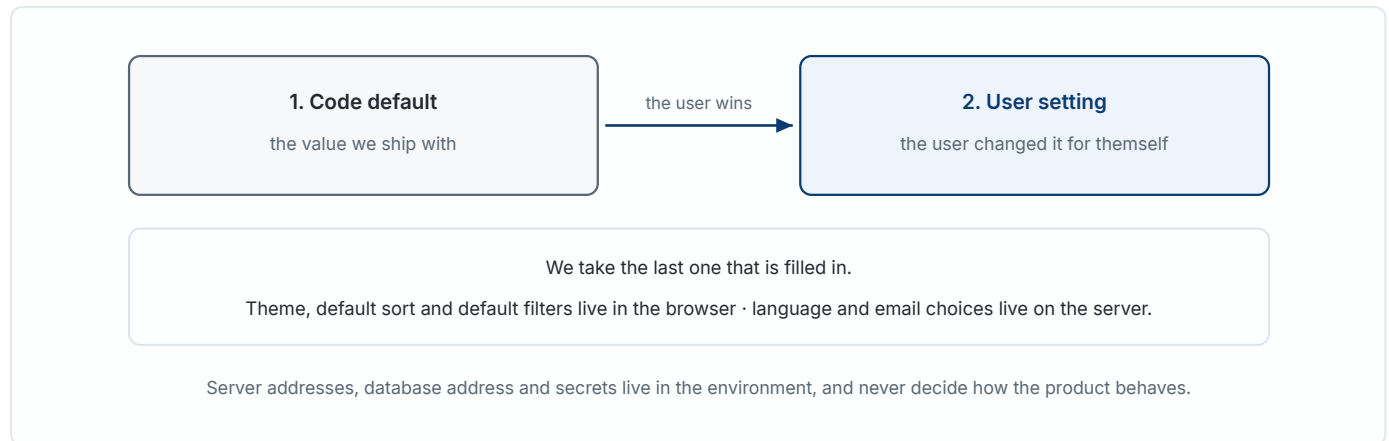
This is the normal path. Only an admin moves a request, and an admin can move it to any status. Because statuses are rows in the database (R-43), an admin can add new ones or retire old ones, and this picture changes with them.

10.2 How the app starts



Only one request before the app appears. Not a chain of calls waiting for each other. If this call fails, we show an error with a Retry button — not a blank page. This is R-52 and hard part H-4.

10.3 How we pick the value of a setting



A user can only change their own settings: theme, default sort and default filters in this browser; language and email choices on the server. A user can never change a setting that belongs to the admin (R-60).

Server addresses, database address and secrets live in the environment, not in the database, and are never used to decide how the product behaves (R-53).

11. Settings and where they live

Where	Who changes it	Examples
Environment	The person who runs the servers	Database address, Redis address, identity provider address, allowed web addresses, mail settings, log level, secrets.
Admin settings (one row in the database)	Admin, while the app runs	Sign-up rule, allowed email domains, comments need approval, the six rate-limit numbers, and the comments switch.
My settings (one row per person)	The person	Language, email choices.
Browser (this device only)	The person	Theme, default sort, default filters.

Feature switches.

Switch	What happens when the admin turns it off
<code>feature_comments_enabled</code>	The comment box and the whole thread disappear. Comment counts disappear from the board. And the server refuses to save a comment, with a message saying comments are off. Screen and server both (R-42).

A switch that only hides a button is not a feature switch. If the server still accepts the action, the switch does nothing. This is hard part H-5.

12. The data

Nine tables. Every table has an id. Every date is kept in UTC. PK means the key of the row, FK means it points at another table, UQ means no two rows can be the same.

12.1 users

A copy of the person we get from the identity provider. We keep no password here.

Field	Kind	Rules	What it is
id	id	PK	Our own id. All other tables point at this.
external_id	text	needed, UQ	The id the identity provider gave. This is how we know it is the same person.
email	text	needed, UQ	Used for the domain rule and for emails. Never shown to other people.
email_verified	yes/no	needed	Only a checked email counts for the domain rule (R-67).
display_name	text	needed, 1-80	Shown on requests and comments.
avatar_url	text	can be empty	Empty means we draw initials.
role	text	user or admin	Checked again on every admin action (R-8).
status	text	active or deleted	deleted means the account was wiped (R-61).
created_at , updated_at , deleted_at	date		When it was made, changed, wiped.

12.2 user_settings

One row per person, holding only what they changed. An empty field means "I did not change this, use the code default".

Field	Kind	Rules	What it is
user_id	id	PK, FK → users	One row per person at most. Goes away with the person.
language	app_language	en , ar , can be empty	Empty = use the code default.
notify_on_comment	yes/no	not null, default true	Comments on my requests.
notify_on_status_change	yes/no	not null, default true	Status changes on my requests.
updated_at	date	needed	When the row last changed.

Why separate columns and not one blob: an empty column says clearly "not changed", every field keeps its own rule and its own type in the database, and picking the value stays one simple line (R-51).

Language is kept on the server, not in the browser, because the email is written at the moment the event happens — someone comments on my request while I am signed out, and the server must know my language then. Theme and sort are kept per device on purpose: they do not follow a person to another browser, another device or a private window.

12.3 `app_settings` — one row only

Field	Kind	Rules	What it is
<code>id</code>	number	PK, always 1	Only one row can ever exist.
<code>registration_policy</code>	text	<code>open</code> / <code>invite_only</code> / <code>domain_restricted</code>	R-67.
<code>allowed_email_domains</code>	list of text	not empty for the domain rule	Kept in small letters.
<code>comments_require_approval</code>	yes/no	needed	R-40.
<code>signup_limit_count</code>	number	1 or more	New accounts, for the whole app. Default 20.
<code>signup_limit_minutes</code>	number	1 or more	In how many minutes. Default 60.
<code>submission_limit_count</code>	number	1 or more	New requests, for one person. Default 10.
<code>submission_limit_minutes</code>	number	1 or more	In how many minutes. Default 60.
<code>vote_limit_count</code>	number	1 or more	Votes and un-votes, for one person. Default 100.
<code>vote_limit_minutes</code>	number	1 or more	In how many minutes. Default 60.
<code>feature_comments_enabled</code>	yes/no	not null, default true	The one feature switch (R-42).
<code>updated_at</code>	date	needed	When the row last changed.

12.4 `categories`

Field	Kind	Rules	What it is
<code>id</code>	id	PK	
<code>name</code>	text	needed, UQ, 1–40	Bug, Feature, Improvement, Question — only as a start. The admin owns this list.
<code>slug</code>	text	needed, UQ	Short name for the web address and filters.
<code>color</code>	text	needed	Always shown with the text too, never colour alone (R-111).
<code>description</code>	text	can be empty	Small help text in the picker.
<code>is_active</code>	yes/no	needed	No = retired (R-45).
<code>created_at</code> , <code>updated_at</code>	date		The list is read in <code>created_at</code> order.

12.5 statuses

Field	Kind	Rules	What it is
id	id	PK	
name	text	needed, UQ, 1–40	New, Under Review, Planned, In Progress, Done, Declined — only as a start.
slug	text	needed, UQ	
color	text	needed	
is_default	yes/no	only one row can be yes	The status every new request gets (R-11). The database makes sure there is only one.
is_active	yes/no	needed	Retired. The default one can never be retired (R-48).
created_at , updated_at	date		The list is read in <code>created_at</code> order, which is pipeline order because the seed data is made that way.

12.6 feedback_requests

The main table. The vote count and comment count are not here. They are counted when we read, so they can never be wrong.

Field	Kind	Rules	What it is
id	id	PK	
title	text	needed, 5–120	R-12.
description	text	needed, 10–5000	Plain text. Line breaks are kept. No formatting is read.
category_id	id	needed, FK, cannot be deleted	The database refuses to delete a used category (R-46).
status_id	id	needed, FK, cannot be deleted	Set by the server, changed only by an admin.
author_id	id	needed, FK, cannot be deleted	Because deleting an account wipes the name but keeps the content (R-61).
is_pinned , pinned_at	yes/no, date		Admin only. The date keeps pinned ones in a fixed order.
created_at , updated_at	date	needed	<code>updated_at</code> changes only when the text changes — not on a vote or a comment.

12.7 votes

Field	Kind	Rules	What it is
id	id	PK	
request_id	id	needed, FK, goes with the request	Delete the request, the votes go too.
user_id	id	needed, FK, goes with the person	

Field	Kind	Rules	What it is
<code>created_at</code>	date	needed	

`request_id` + `user_id` together must be unique. This one line is what makes R-26 true, even with ten clicks in the same second. Un-voting is deleting the row.

12.8 comments

Field	Kind	Rules	What it is
<code>id</code>	id	PK	
<code>request_id</code>	id	needed, FK, goes with the request	
<code>author_id</code>	id	needed, FK, cannot be deleted	Same reason as the request author.
<code>body</code>	text	1-2000; empty when deleted	Plain text.
<code>state</code>	text	<code>published</code> / <code>pending</code> / <code>deleted</code>	<code>pending</code> only when approval is on. <code>deleted</code> is the grey line (R-38).
<code>created_at</code> , <code>updated_at</code> , <code>deleted_at</code>	date		<code>updated_at</code> changes when the writer edits it.

12.9 invitations

Only used when the sign-up rule is invite-only.

Field	Kind	Rules	What it is
<code>id</code>	id	PK	
<code>email</code>	text	needed, UQ	A name on a list. An invite email is sent to this address.
<code>accepted_at</code>	date	can be empty	Filled in when that email signs in for the first time.
<code>created_at</code>	date	needed	

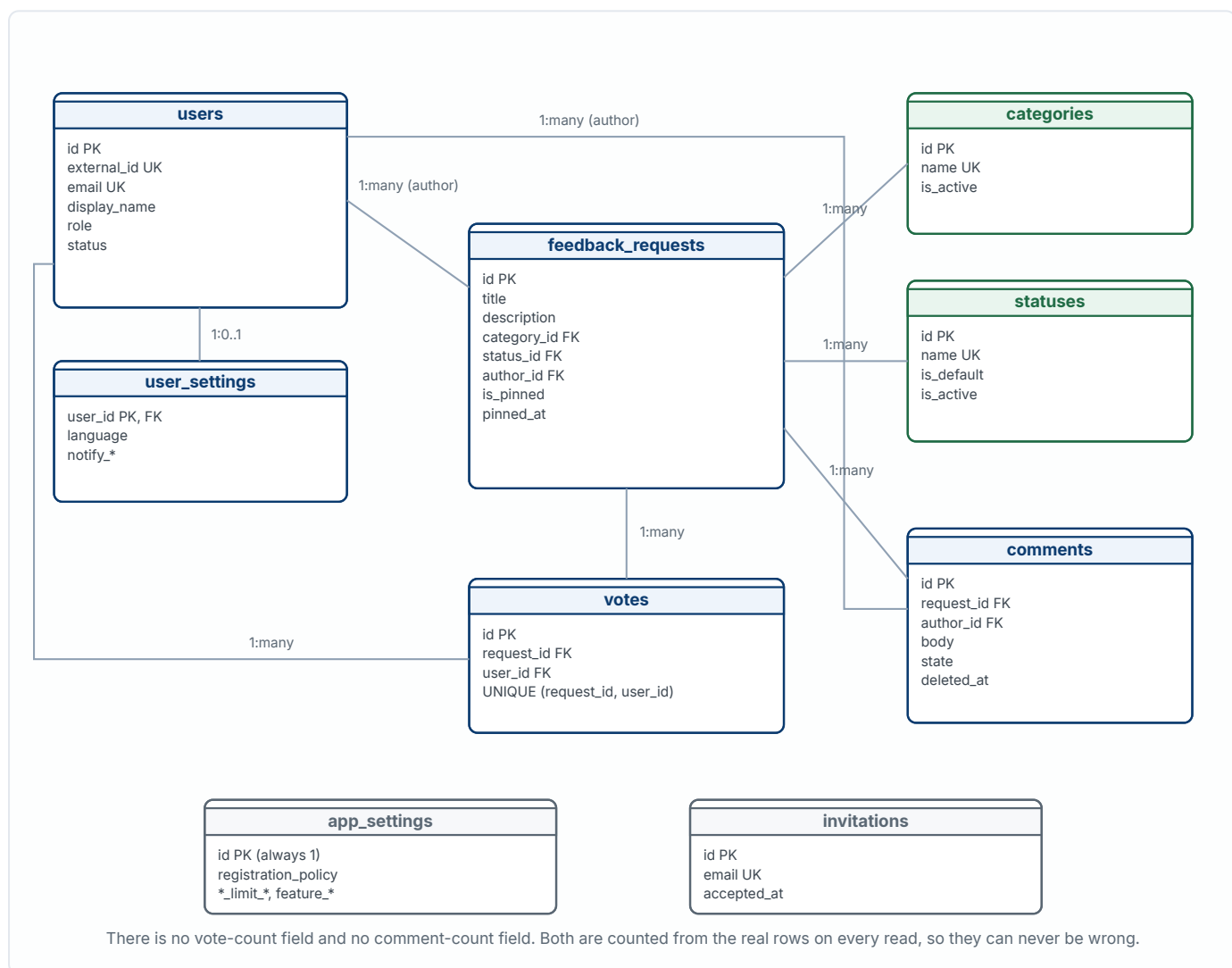
12.10 How the tables connect

Link	How many	If the parent goes	Why
<code>users</code> → <code>user_settings</code>	1 to 0 or 1	goes too	Settings mean nothing without the person.
<code>users</code> → <code>feedback_requests</code>	1 to many	blocked	Deleting an account wipes the name, keeps the content (R-61).
<code>users</code> → <code>comments</code>	1 to many	blocked	Same.
<code>users</code> → <code>votes</code>	1 to many	goes too	A vote is a personal signal, not content.

Link	How many	If the parent goes	Why
<code>categories</code> → <code>feedback_requests</code>	1 to many	blocked	Retire, never delete (R-46).
<code>statuses</code> → <code>feedback_requests</code>	1 to many	blocked	Same.
<code>feedback_requests</code> → <code>votes</code>	1 to many	goes too	Votes belong to the request.
<code>feedback_requests</code> → <code>comments</code>	1 to many	goes too	Same.

`invitations` has no foreign key: `invitations.email` matches `users.email`, and both are unique.

13. The ERD



Blue = the main work. Green = the two lists the admin owns. Grey = settings and helpers, which have no links to draw. Only the key fields are shown; part 12 has all of them.

The two counts are missing on purpose. If counting them on every read ever becomes too slow, a counter can be added behind the same repository interface (R-147) — a small change, not a new design.

14. Rules that are always true

These must hold no matter which screen or which button caused it. Where the database is what stops it, we say so — a check in code and a check in the database are not the same strength when two things happen in the same second.

Promise	Who stops it	Rules
Only people the sign-up rule and the sign-up limit allow get an account.	Server	R-4, R-67, R-130
Only a checked email counts for the domain rule.	Server	R-67
Changing the sign-up rule never removes people who already got in.	Server	R-4
One person, one vote per request.	Database	R-26
The vote count always equals the real number of votes.	By design — there is no count field	R-28
Two categories or two statuses can never have the same name.	Database	R-44
Exactly one status is the first one.	Database	R-47
A category or status in use can never be deleted.	Database	R-46
There is always at least one category and one first status, so writing a request is always possible.	Server	R-48
A person cannot send more requests or votes than allowed. Two in the same second cannot both slip through.	Server, inside one database step	R-69, R-130, R-132
Only the writer changes their own words. An admin deletes, never rewrites.	Server	R-13, R-35, R-36
Who owns a thing is read from the database, never from what the browser sent.	Server	R-7
A setting is picked as: code default, then user. Always in that order, in one place.	Server	R-51
A person can never change an admin setting.	Server	R-60
A feature switch that is off blocks the screen and the server.	Both	R-42
A missing or broken switch falls back to the code default, so a bad settings row can never switch something on by accident.	Server	R-42
Nobody is emailed about their own action, and only if they asked for it.	Server	R-71
An email problem can never undo a comment.	Server	R-72
Deleting an account keeps the writing and drops the name.	Server	R-61

Promise	Who stops it	Rules
The app can never be left with no admin.	Server	R-62
No module reads another module's tables.	Lint, in CI	R-141, R-143

15. Every case on every screen

The normal case is the easy one. The other four are where the marks are (H-3).

15.1 Board

Case	What the person sees
Normal	The list, with counts, pinned ones on top, page buttons, and the total number found.
Loading	Grey blocks the same size and shape as the rows. Nothing jumps when the real rows arrive.
Empty — no data	"No requests yet. Be the first." With the button to write one. This is the first thing a new company sees, so it should not look broken.
Empty — filters	"Nothing matches these filters." With a Clear filters button. A different message from the one above.
Error	"We could not load the board." With a Try again button. Filters stay as they were.
Edge	Page 5 of a list that shrank to 2 pages → go back to the last real page, do not show an empty page. A search with no results → the filter message, not the error one. A category retired while filtering by it → keep working, show it as retired.

15.2 Request page

Case	What the person sees
Normal	The request, the vote button in the right state, the comments, the comment box.
Loading	Grey blocks for the request and for the comments, loaded separately.
Empty	No comments yet → "No comments yet. Start the discussion."
Error	Request failed to load → error with Try again. Comments failed but the request loaded → error only in the comments part; the rest of the page still works.
Not found	Deleted while it was open, or a bad address → "This request does not exist any more," and a link back to the board.
Not allowed	Someone opens the edit page for a request that is not theirs → clear message, no form.
Edge	Comments switched off → the whole comment part is gone (R-42). Approval on → my own waiting comment is shown to me marked "waiting". A deleted comment → a grey line, the thread keeps its shape. The author deleted their account → "Deleted user".

15.3 New request / edit request

Case	What the person sees
Normal	Three fields and a Save button.
Bad input	A message under the field that says how to fix it. The cursor moves to the first bad field. Nothing is lost.
Over the limit	"You have sent 10 requests in the last hour. You can send another at 14:00." The text they wrote is kept, not thrown away.
Error	Saving failed → the form stays filled in, with a message and a Try again button. Never a blank form.
Edge	The category was retired while the form was open → a clear message asking them to pick another. Two fast clicks on Save → only one request is made. Leaving with unsaved text → ask first.

15.4 Voting

Case	What happens
Normal	Click → the number goes up at once, the button shows "voted".
Take it back	Click again → the number goes down, the button goes back.
Double click	Still one vote. No error message the person has to understand.
Error	The number goes back to what it was, and a small message says it did not save.
Edge	The request was deleted a second ago → a clear message and the person is sent back to the board. Voting on my own request → allowed. Over the vote limit → refused (R-130).

15.5 Comments

Case	What happens
Normal	Write, save, it appears at the top without a reload, the box clears, the count goes up.
More	"Show more comments" adds older ones under it, and the button is gone at the end of the list.
Waiting	Approval is on → only I see mine, marked "waiting for approval". The count does not move.
Error	The text I typed is kept in the box, with a message and a Try again button.
Not allowed	Editing someone else's comment → refused by the server, even if the button was somehow visible.
Edge	A comment arrives while I am reading → it does not push a comment I have already seen into the next block; the cursor stops that (R-33b). Comments switched off while I am typing → a clear message when I press save, not a silent failure. A comment on a request that was just deleted → clear message.

15.6 My settings

Case	What happens
Normal	Each part saves on its own and says "Saved".

Case	What happens
Bad input	Empty name → a message under the field.
Error	The old value comes back on screen, with a message. The person's typing is not lost.
Delete account	A question that says clearly: sign-in stops, posts stay as "Deleted user", votes go. They must type or click to confirm.
Edge	The last admin tries to delete their account → refused with the reason (R-62). A saved default filter points at a retired category → it is skipped, nothing breaks.

15.7 Admin screens

Case	What happens
Normal	Change a status, pin, delete a comment, add a category, change a setting. Everything shows at once.
Not allowed	A normal person types the admin address → the server refuses. The screen is not just hidden.
Blocked on purpose	Retire the first status → refused, with the reason. Retire the last category → refused. Delete a used category → refused, offer to retire instead.
Error	A setting failed to save → the old value stays on screen with a message. No half-saved settings.
Edge	Two admins change the same setting at once → the last one wins. Turning off comments while people are typing → see 15.5.

15.8 Sign in and start-up

Case	What happens
Normal	Sign in, one call, the app appears.
Not allowed to join	The email does not pass the sign-up rule → a clear page saying why. Not a blank screen, not a loop back to sign-in.
Start-up call fails	An error page with a Try again button. Never a spinner forever, never a white page.
Token runs out while using the app	It is renewed quietly. If that fails, back to sign-in, and we remember which page they wanted.
Edge	The identity provider is down → a clear message saying sign-in is not available, not a broken page. An admin changed a setting a minute ago → the browser picks it up on its next load, not instantly. The sign-up limit is full → a person who signed in fine still cannot join yet, and is told to try later.

16. Things that are not features

These are not buttons. They are how the whole app must behave.

16.1 Safety

#	Rule
R-93	Every check that decides "can this person do this" runs on the server, on the saved row. Hiding a button is only a kindness.
R-94	Knowing an id gives nothing extra. Person A cannot reach person B's things by typing the id.
R-95	Everything sent to the server is checked against a written shape. Unknown fields are refused, not ignored. Every text and every number has a smallest and a biggest value.
R-96	Limits on the server: at most 1 MB per request, and a rough limit per computer that guards every save. Both come from the environment, and are separate from the admin's own rate limits (R-130).
R-97	Nothing a person types is ever put straight into a database query. Sort names and filter names come from a fixed list; every other value is a bound parameter.
R-98	What people write is shown as plain text. We never read it as HTML and never as formatting. This removes a whole family of attacks in one line.
R-99	Another person's email or settings are never sent. On a request and a comment we send only the writer's name and picture (see R-77).
R-100	An error message never shows the inside of the system: no stack, no database text, no library names. The full story goes to the log, with an id that is also given to the person so support can find it.
R-101	Normal safety headers are set. Only our own web addresses are allowed to call the API.
R-102	No secret is ever put in the repository. Only an example file with fake values. Containers do not run as root.

16.2 Speed

For the size in part 1, on a normal development machine, not counting the first cold start.

#	Rule
R-103	The board — search, filter, sort, page and counts — under about 300 ms for almost every call.
R-104	A request page, and saving a vote or a comment: under about 250 ms.
R-105	The start-up call: one call, under about 200 ms. This is a promise about the shape, not only the speed.
R-106	The database has an index for every filter and every sort we offer, and one for the newest-first comment read, <code>comments (request_id, created_at, id)</code> . The search is a scan over the title and the description. No core screen reads the whole table.
R-107	The first paint is quick, and nothing on the page jumps when the data lands.

16.3 Everyone can use it

#	Rule
R-108	We aim at WCAG 2.2 AA. Anything we know we did not reach goes in the README instead of being left to be found.
R-109	Everything works with the keyboard only. You can always see where you are. A skip-to-content link at the top.
R-110	Every control has a name a screen reader can say. A control with a state — the vote button, a filter — says its state too, not only its name.
R-111	Colour is never the only way to tell something. A status and a category always show their text next to their colour. Text has enough contrast in both light and dark.
R-112	Things that finish in the background are read out. When a form fails, the cursor goes to the first bad field.
R-113	The page still works at 200% zoom and down to a 320 pixel wide screen, with no sideways scrolling.
R-114	If the computer asks for less movement, we use less movement.

16.4 Running it and keeping it right

#	Rule
R-115	Important promises are kept by the database, not only by code: one vote per person, links between tables, only one first status.
R-116	Side jobs that are not part of what the person asked for — emails — can never break what they asked for.
R-117	The same action done twice at the same moment ends in one result, not an error and not a double.
R-118	The API keeps nothing in its own memory between calls, so any copy of it can answer any call. Shared state lives in Postgres or Redis (R-149).
R-119	Logs are machine readable, with an id that follows one call through the system. Never a password, a token, or a person's private data in a log.
R-120	The example data can be loaded again and again without breaking, and it is rich enough to see every case in part 15 — including the empty ones.

16.5 Keeping the code readable

#	Rule
R-121	One way to do each thing — see R-150 and the rules under it.
R-122	Strong types everywhere. No untyped holes at the edges of the app.

#	Rule
R-123	Each part of the code owns its own tables. Other parts ask it, they do not touch its tables (R-141).
R-124	Tests must cover at least: every permission rule with its no case, the input limits, how a setting is picked, voting twice, the three rate limits, and the board's search, filter, sort and pages. How they are written is in 8.7.

17. How we know a feature is done

Written as: *given* a starting point, *when* someone does something, *then* this must be true. These are the sentences the tests check, and under TDD (R-156) they are written before the code.

Sign in

- Given a new person and the open sign-up rule, when they sign in with email and password or with a social account, then an account is made and they land on the board with their name in the corner.
- Given the domain rule allows `example.com`, when someone with a checked `@other.com` email signs in, then they see a clear message and no account is made.
- Given the domain rule, when someone with an unchecked `@example.com` email signs in, then they are refused, and the message says email checking is the reason.
- Given nobody is signed in, when the API is called directly, then it answers 401 — not a redirect, and not data.

Write a request

- Given a signed-in person, when they save a good title, text and category, then the request exists with the first status, their name, zero votes, and it is on the board without a manual refresh.
- Given a four letter title, when they press save, then a message under the field says the smallest length, the cursor goes there, and nothing is saved.
- Given a browser that also sends a status or a vote count, when the server reads it, then those extra fields are refused and the server's own values are used.

Use the board

- Given a board with data, when a word from a description is searched, then the matching requests come back and the search is in the web address.
- Given two statuses and one category are ticked, when the list loads, then only requests that match (status A or status B) and that category are shown, with the total.
- Given filters that match nothing, when the list loads, then the "nothing matches these filters" message appears with a Clear button — not the "no requests yet" message.
- Given a pinned request, when a filter hides it, then it is not shown; when a filter includes it, then it is first.
- Given a browser asking for a made-up sort name, then the server refuses the sort name.

Vote

- Given a request I have not voted for, when I use the button with the mouse or the keyboard, then the count goes up by one and the button shows I voted.
- Given I already voted, when I use it again, then my vote is taken back and the count goes down by one.
- Given ten votes sent at the same moment by the same person, when they are all handled, then exactly one vote exists and no call fails with a confusing error.
- Given the server refuses, when the screen puts the number back, then the old number is shown with a short message.

Comment

- Given a signed-in person, when they save a comment, then it appears at the top with no reload, the box clears, and the count goes up.
- Given a request with 60 comments, when I open it, then the newest comment is the first one on the page.
- Given I am reading the first block and someone else adds a comment, when I press "Show more comments", then no comment is shown twice and none is missed.
- Given someone else's comment, when a normal person tries to edit it through the API, then the server answers 403 and nothing changes.
- Given a comment in the middle, when its writer deletes it, then a grey line takes its place, the order stays, and the count goes down.
- Given approval is on, when a person saves a comment, then only they and admins see it, marked as waiting, and it is not counted until an admin approves it.

My own things

- Given my own request, when I change the text, then it is saved.
- Given someone else's request, when a normal person calls edit or delete with its id, then the server answers 403 and nothing changes — whatever the screen showed.
- Given my own request with votes and comments, when I delete it after confirming, then it and its votes and its comments are gone from the board.

Admin work

- Given an admin, when they change a status, then the new status shows everywhere the request appears.
- Given an admin, when they pin a request, then it is first for everyone, still following the filter rule above.
- Given a normal person, when they call any admin action, then the server answers 403 — whether or not the screen ever showed it to them.
- Given a normal person, when they call the invite endpoint by hand, then the server answers 403 and nothing is saved.
- Given an admin, when they delete a rude comment, then it becomes a grey line.

The feature switch

- Given comments are on, when I open a request, then the thread and the box are there and counts show on the board.
- Given an admin turns comments off, when I next load the app, then the box and the thread are gone and counts are gone everywhere.
- Given comments are off, when a browser sends a comment straight to the API, then the server answers 403 with a message saying comments are switched off. This is the test that proves the switch is real.

Categories and statuses

- Given an admin adds a category, when people next load the app, then they can choose it.
- Given a category used by old requests, when the admin retires it, then it is gone from the picker and those old requests still show it correctly.
- Given the first status, when an admin tries to retire it, then it is refused with the reason.
- Given one status is the first one, when the admin makes another one first, then exactly one first status exists afterwards.

Settings and start-up

- Given a fresh load, when we watch the network, then there is exactly one settings call before the app shows — not a chain.
- Given my theme is dark and the code default is light, when I load the app, then it is dark, with no white flash first.
- Given a browser trying to change an admin setting through the user API, then it is refused and nothing is saved.
- Given the start-up call fails, when the app starts, then an error with a Try again button is shown — not a white page and not an endless spinner.
- Given I set dark on my laptop, when I open the app on my phone, then it is the code default — not dark.
- Given I clear the browser data, when I load the app, then theme, default sort and default filters go back to the code default.
- Given a body that tries to set the theme through the user settings API, then it is refused with a message and nothing is saved (R-60).

The three rate limits

- Given a limit of 3 per hour and a person who sent 3, when they send a fourth, then it is refused, the message names the time they may try again, and nothing is saved.
- Given the same person an hour later, when they send one, then it works.
- Given a person at the request limit who deletes one of their own requests, when they send a new one straight away, then it is still refused.
- Given the sign-up limit is 20 in 60 minutes and 20 accounts were just made, when a twenty-first person signs in at the identity provider, then no record is made, they see the "cannot join" page, and the message says to try later.

- Given the vote limit is reached, when the person votes or un-votes once more, then it is refused and the vote count does not change.
- Given the admin raises any of the three limits, when the person tries again, then it works, with no restart.

Emails

- Given `notify_on_comment` is off, when someone comments on my request, then no email is sent.
- Given I comment on my own request, then no email is sent to me.
- Given comments need approval, when a comment is written, then no email is sent; when the admin approves it, then the email is sent.
- Given comments need approval, when the admin rejects the comment, then no email is ever sent.
- Given the mail server is down, when someone comments, then the comment is still saved and the answer is not slower.
- Given an admin adds an invitation, then an email with a sign-up link is sent to that address.
- Given my language is Arabic and someone comments on my request while I am signed out, then the email is sent in Arabic.

Profile and account

- Given I change my name, then it changes on my old requests and comments too.
- Given my theme follows the computer, when the computer switches to dark, then the app switches too, with no reload.
- Given I change the language, then the screen changes at once, and any missing word shows in English, not as a code word.
- Given I delete my account after a question that said my posts will stay, then my sign-in stops working, my posts show "Deleted user", and my votes are gone.
- Given I am the only admin, when I try to delete my account, then it is refused with the reason.

Architecture and code

- Given the repository, when the dependency check runs, then no module imports another module's internals and no `domain` file imports a framework or the ORM.
- Given an HTTP response, when it is compared with its DTO, then no ORM type and no field the endpoint did not mean to send appears in it.
- Given the API, when two copies of it run behind a load balancer, then any copy can answer any call — no state is kept in the process.

Running it

- Given a fresh copy of the repository on a machine with only Docker, when the written command is run, then the front end, API, worker, database, Redis, identity provider and mail catcher all start, the database is set up, example data is loaded, and a written test account can sign in — with no secret extra step.
- Given the Kubernetes files, when they are applied to a small local cluster, then the migration step finishes before the app is marked ready, and the health checks tell the truth.

- Given the repository, when it is searched for secrets, then there are none — only example values.

Keyboard and small screens

- Given only a keyboard, when I go through the board, open a request, vote, comment and open a pop-up, then every step works, I can always see where I am, and the pop-up gives the keyboard back when it closes.
- Given a 360 pixel wide screen, when I open any page, then it fits with no sideways scrolling and every control still works.
- Given a form that fails, when I press save, then the problem is read out, the cursor goes to the first bad field, and each message says how to fix it.

Remember what this file is. It says what the app must do. It does not say what is built. When we start, `README.md` is the place that tells the truth about what works. If we drop a rule from this file, the reason goes in `SCOPE.md` — we do not quietly delete it here.